

HW6

12.30

T1

- PUSH A, PUSH B, POP, PUSH C, PUSH D, POP, PUSH E, POP, POP, PUSH F

- (a) What does the stack contain after the PUSH F?

A AB A AC ACD AC ACE AC A AF

- (b) At which point does the stack contain the most elements?

PUSH D

PUSH E

- (c) As for queue

A AB B BC BCD CD CDE DE E EF

T2

T2

Two students wrote interrupt service routines for an assignment. Both service routines did exactly the same work, but the first student accidentally used `RET` at the end of his routine, while the second student correctly used `RTI`. There are three errors that arose in the first student's program due to his mistake. Describe any two of them.

- `RET:JMP R7`

The `RTI` instruction (opcode = **1000**, with no operands) pops the top two values on the system stack into the PC and PSR. Since the PC contains the address following the address of the `TRAP` instruction, control returns to the user program at the correct address.

Finally, once the PSR has been popped off the system stack, `PSR[15]` must be examined to see whether the processor was running in User mode or Supervisor mode when the `TRAP` instruction was executed. If in User mode, the stack pointers need to be adjusted to reflect that now back in User mode, the relevant stack in use is the user stack. This is done by loading the `Saved_SSP` with the current contents of `R6`, and loading `R6` with the contents of `Saved_USP`.

T2

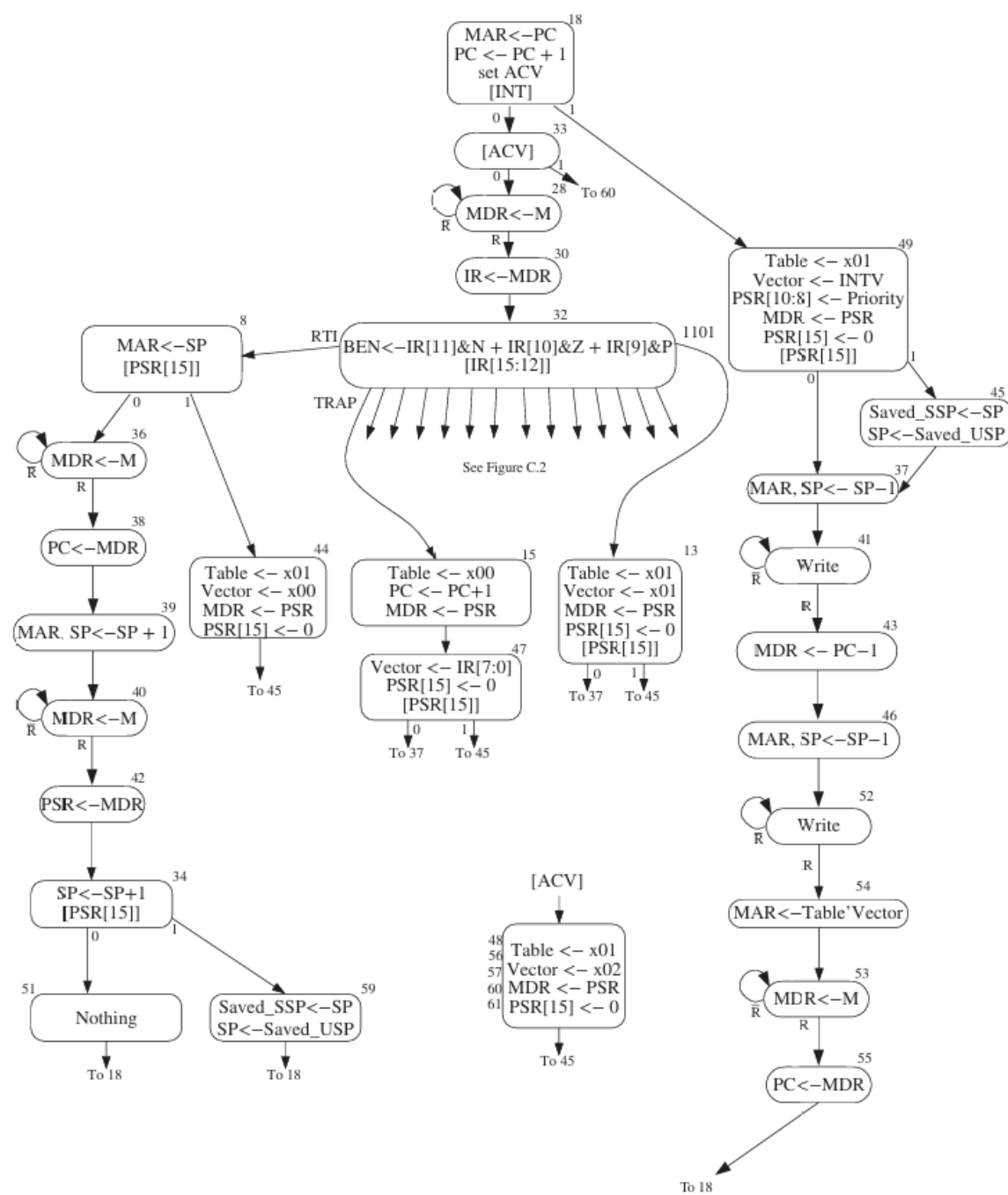


Figure C.7 LC-3 state machine showing interrupt control.

T2

- 因为是中断服务，所以 R7 不会保存返回地址，RET 会跳转到错误的位置
- 直接调用 RET 没有恢复保存下来的 Processor Status Register，因此程序没有从特权模式恢复到用户模式，之前保存的状态码也没有恢复
- RET 不涉及对栈的操作，但从中断服务中恢复，应该要恢复栈指针

T3

- (a) What problem could occur if a program does not check the ready bit of the KBSR before reading the KBDR?

程序可能反复读取同一个字符

- (b) What problem is likely to occur if the keyboard hardware does not check the KBSR before writing to the KBDR?

KBDR 存的数据可能还没有被读取，用户输入字符可能丢失

- What problem could occur if the display hardware does not check the DSR before writing to the DDR?

display hardware 并不会写入 DDR

what about a program does not check the DSR before writing to the DDR?

T4

Assemble the following LC-3 assembly language program

```
1      .ORIG x3000
2      AND R0, R0, #0
3      ADD R2, R0, #10
4      LD  R1, MASK
5      LD  R3, PTR1
6  LOOP LDR R4, R3, #0
7      AND R4, R4, R1
8      BRz NEXT
9      ADD R0, R0, #1
10     NEXT ADD R3, R3, #1
```

统计 x4000 开始的
十个数据中首位为1
(负数) 的个数, 存
在地址 x5000 处

主要看MASK x8000
和AND 操作。

```
11     ADD R2, R2, #-1
12     BRp LOOP
13     STI R0, PTR2
14     HALT
15     MASK .FILL x8000
16     PTR1 .FILL x4000
17     PTR2 .FILL x5000
18     .END
```

What does the program do?

T5

The following code implements the function of reversing a string. Please complete the missing parts.

```
1      .ORIG x3000
2      LEA  R6, STACKBASE
3      LEA  R0, PROMPT
4      TRAP x22
5      AND  R1, R1, #0
6  LOOP    TRAP x20
7          TRAP x21
8          ADD  R3, R0, #-10
9          BRz  INPUTDONE
10         JSR  PUSH
11         ADD  R1, R1, #1
12         BRnzp LOOP
13  INPUTDONE  ADD  R1, R1, #0
14          BRz  DONE
15  LOOP2    ---
16          TRAP x21
17          ADD  R1, R1, #-1
18          BRp  LOOP2
19  DONE     TRAP x25
20
21  PUSH     ---
22          ---
23          ---
24  POP      LDR   R0, R6, #0
25          ADD  R6, R6, #2
26          RET
27  PROMPT   .STRINGZ "Please enter a sentence:"
28          STACKSPAC .BLKW #50
29          STACKBASE .FILL #0
30          .END
```

(a) Complete the code for the subroutine `PUSH`

(b) What instruction should be filled in line 15 ? Also, explain the role of the R1 register in the code.

- (a)参考POP: LDR换STR, +2换-2:
ADD R6, R6, #-2
STR R0, R6, #0
- (b)先入后出反向输出字符串:
JSR POP

R1 寄存器的作用是记录输入字符串的长度, 便于后续逆序输出

T6

In this question, we modify how **JSR** / **JSRR** and **RET** works in LC3.

JSR / **JSRR** will use the stack as the linkage instead of R7 by pushing the return PC onto the stack. Similarly, **RET** will load the PC with the value popped from the top of the stack.

In memory, there are 10 subroutines to print the 10 digits. For example, the subroutine to print the number “5” consists of the following instructions:

```
1  Print5      LD R0 Lable5
2              OUT
3              RET
4  Lable5      .FILL x35
```

Since each subroutine requires 4 memory locations, the starting address of each subroutine is always 4 greater than the starting address of the previous subroutine. The starting address of the subroutine Print0 is x5000. Therefore, the starting address of the subroutine Print1 is x5004, the starting address of the subroutine Print2 is x5008, and so on.

A student wishes print number “306” with a program that does not use any **JSR** / **JSRR** instructions nor I/O trap service routines in the main program.

Your Job: Fill in the blanks in the main program, so that executing this program will result in printing the digits “306” on the console, and then halt the machine.

Hint: What happens when a **RET** is executed without a JSR being executed beforehand ?

```
1      .ORIG x3000
2      LD  R6, _____
3      LD  R0, _____
4      ADD _____
5      STR R0, R6, #0
6      LD  R0, _____
7      ADD _____
8      STR R0, R6, #0
9      LD  R0, _____
10     ADD _____
11     STR R0, R6, #0
12     LD  R0, _____
13     ADD _____
14     STR R0, R6, #0
15     RET
16     _____
17     _____
18     _____
19     _____
20     HALT
21     SP_INIT .FILL xFE00
22     .END
```

T6

```
1      .ORIG x3000
2      LD  R6, SP_INIT
3      LD  R0, LABELH
4      ADD R6, R6, #-1
5      STR R0, R6, #0
6      LD  R0, LABEL6
7      ADD R6, R6, #-1
8      STR R0, R6, #0
9      LD  R0, LABEL0
10     ADD R6, R6, #-1
11     STR R0, R6, #0
12     LD  R0, LABEL3
13     ADD R6, R6, #-1
14     STR R0, R6, #0
15     RET
16 LABELH .FILL x3012
17 LABEL6 .FILL x5018
18 LABEL0 .FILL x5000
19 LABEL3 .FILL x500c
20     HALT
21 SP_INIT .FILL xFE00
22     .END
```

代码的逻辑是把需要调用的子程序地址预先存入栈中，RET直接依次调用，注意 LABELH 的值是代码中 HALT 指令的地址，直接从 x3000 数行数得来

T7

The following program copies a specified number(SIZE) of memory values from a source(SRC) to a destination(DST).

```
1      .ORIG x300
2      LD  R2, DST
3      LD  R1, SRC
4      LD  R0, SIZE
5  LOOP BRz EXIT
6      ADD R0, R0, #-1
7      ADD R4, R1, R0
8      LDR R5, R4, #0
9      ADD R4, R2, R0
10     STR R5, R4, #0
11     ADD R0, R0, #0
12     BR LOOP
13  EXIT HALT
14  SRC  .FILL x4000
15  DST  .FILL x5000
16  SIZE .FILL x10
17     .END
```

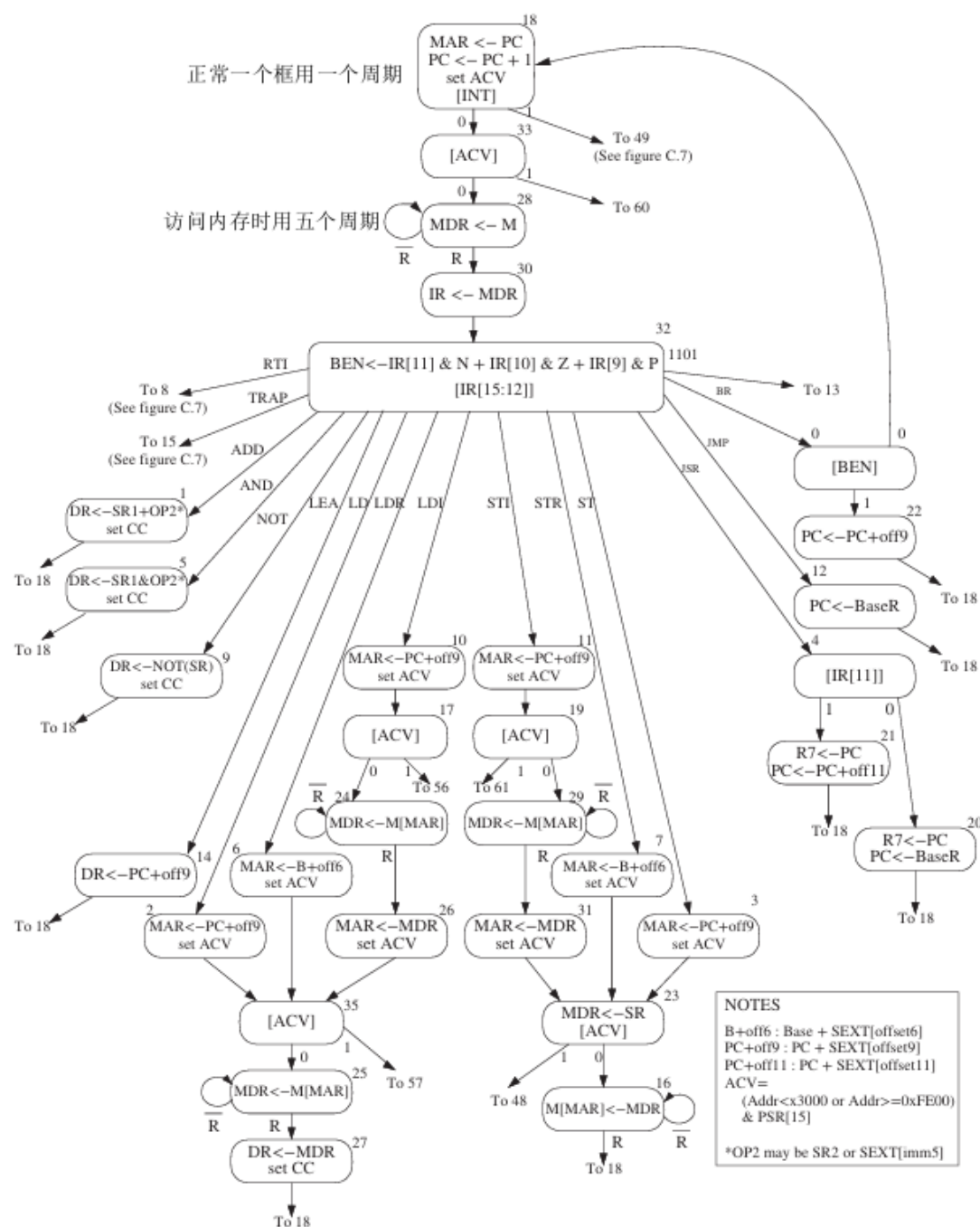
Do not count the HALT instruction when answering the questions.

(a) How many instruction are processed during the runtime period ?

(b) Assuming one memory access takes 10 clock cycles, how many clock cycles will the program take ?

$$(a) 3*1 + (16+1)+7*16=132$$

T7



(b) 计算指令的周期：数框框，内存访问 (R) *10，否则*1

T7

(a) 注意 x10 是十进制的 16 所以循环次数是16次，开局三条指令+最后一次跳转： $8 \times 16 + 3 + 1 = 132$

(b) $144 \times 16 + 3 \times 27 + 16 = 2401$

指令	指令周期
LD	27
ADD	15
LDR	27
STR	26
BR (跳转)	16
BR (不跳转)	15

循环期间：1 个不跳转的 BR，1 个跳转的 BR，4 个 ADD，1 个 LDR，1 个 SDR

则一次循环的指令数： $16 + 15 + 4 \times 15 + 27 + 26 = 144$